

ORIGINAL RESEARCH

Collaborative decision-making for UAV swarm confrontation based on reinforcement learning

Yongkang Jiao^{1,2} | Wenxing Fu¹ | Xinying Cao² | Qiangqing Su³ | Yusheng Wang⁴ | Zixiang Shen⁴ | Lanlin Yu⁴ 

¹Unmanned System Research Institute, Northwestern Polytechnical University, Xi'an, China

²Aeronautical Operation Institute Naval Aeronautical University, Yantai, China

³People's Liberation Army of 91811, China

⁴School of Electrical Engineering and Automation, Hefei University of Technology, Hefei, China

Correspondence

Wenxing Fu, Unmanned System Research Institute, Northwestern Polytechnical University, Xi'an, China.
Email: wenxingfu@nwpu.edu.cn

Abstract

With the advancement of unmanned aerial vehicle (UAV) technology, research on adversarial interactions within UAV swarms has gained significant attention domestically and internationally. However, the existing decision-making algorithms are primarily tailored to homogeneous UAV swarm adversarial scenarios, facing challenges such as complex reward function design and limited decision-making timeliness when applied to more intricate scenarios. This article investigates the real-time control decision-making issues in UAV swarm adversarial interactions. First, an adversarial simulation environment for UAV swarms is constructed, which effectively unifies the environment and state representation, enhancing the response speed of our UAVs. Second, a distributed UAV swarm collaborative control algorithm based on multi-agent reinforcement learning is proposed, and an effective sparse reward function is designed to guide UAVs in adversarial gaming, making the UAV strategies more aggressive, enhancing the adversarial intensity, and further optimizing the control strategy to meet real-world demands better. Finally, the real-time performance and scalability of the proposed method are validated through simulations.

1 | INTRODUCTION

Unmanned Aerial Vehicles (UAVs), commonly known as drones, are aircrafts that can be either remotely controlled or operate autonomously. As they do not need a pilot to be on board, UAVs offer a multitude of advantages. Some of these are flexible operation, accurate positioning, miniaturization, intelligence, high reliability, strong adaptability, high speed, and low cost. Under the control of automated systems, UAVs can perform a diverse range of flight missions and are extensively used in fields like the military, commercial activities, agriculture, and rescue operations [1–6]. However, a single UAV has numerous limitations when performing tasks. For instance, there is a lack of backup and support, an inability to cover large areas, and a lack of diversity. As a result, it is difficult for a single UAV to complete complex tasks independently. Nevertheless, the comprehensive upgrade of combat systems requires UAVs to operate in swarms in order to adapt to the complexity of modern battlefield environments. UAV swarm confrontation [7–9] is one of the most popular applications of intelligent UAVs. In

this case, multiple UAVs form a swarm to carry out coordinated military operations [10].

A swarm composed of multiple types of UAVs offers many advantages, including distributed confrontation, enhanced offensive capabilities, information sharing, flexibility, resilience, and risk dispersion [11–14]. These advantages enhance overall defense capabilities and provide stronger firepower coverage, making UAV swarms better suited for complex combat environments. However, traditional manual remote control methods are constrained by various factors including communication range, weather conditions, and electromagnetic interference, making them unsuitable for complex and dynamic operational scenarios. Therefore, improving the intelligence of UAVs, enabling them to make autonomous decisions based on situational factors, is a key direction for future development. This article explores the technical mechanisms and implementation approaches for enhancing UAV intelligence and discusses its potential applications in swarm confrontations.

The research methods employed in the field of UAV confrontation can be broadly categorized into three main

This is an open access article under the terms of the [Creative Commons Attribution-NonCommercial-NoDerivs](https://creativecommons.org/licenses/by-nc-nd/4.0/) License, which permits use and distribution in any medium, provided the original work is properly cited, the use is non-commercial and no modifications or adaptations are made.

© 2025 The Author(s). *IET Control Theory & Applications* published by John Wiley & Sons Ltd on behalf of The Institution of Engineering and Technology.

approaches: game-theory-based methods [7, 15–18], optimization techniques [8, 19], and artificial intelligence (AI) methods [11–14, 20, 21]. These diverse research methodologies have been leveraged to address the challenges and complexities of UAV swarm confrontation. Representative methods based on game theory include differential games [7, 15], influence diagrams [16], and matrix games [17, 18], which have been widely utilized to analyze and model strategic interactions among UAV swarms and their adversaries. A differential game mathematical model for tackling aerial combat issues has been constructed in [7], using non-integral and relative motion to find the most resistant trajectory in [15], although real-time performance is not guaranteed. Using a hierarchical game theory approach, distributed adaptive control for large-scale multi-group multi-agent systems effectively reduces communication traffic and computational complexity while ensuring system stability and adaptability in dynamic environments; however, the complexity of the solution process makes it challenging to extend to complex environments. The mathematical model in [16] uses multi-stage influence diagrams to depict the decision-making process and address sequential decision-making in one-on-one aerial combat, but its applicability is limited for UAV swarm combat scenarios. The algorithm described in [17] provides a dimensionality reduction-based matrix game-solving approach to promptly resolve strategies, meeting real-time demands for UAV combat decision-making. However, effectively extending these approaches to address large-scale UAV combat problems poses challenges. Through the analysis of the payoff matrix, the benefits and costs of different strategy combinations are evaluated, and the local stability of the system is determined using the Jacobian matrix, identifying the evolutionarily stable strategy (ESS) in [18], but real-time performance is not guaranteed.

On the other hand, methods based on optimization theory transform UAV combat decision-making problems into optimization problems, which can then be solved to select the optimal action as the output. These methods utilize techniques such as Bayesian inference [8] and genetic algorithms [19] to formulate and solve the optimization model. The autonomous maneuver decision-making system for UAV aerial combat scenarios, as proposed in [8], effectively utilized Bayesian inference theory. However, its capability to address large-scale UAV combat problems is constrained. The combination of knowledge-based aerial combat action libraries and genetic algorithms in reference [19] aimed to reduce computational complexity. Nevertheless, its extension to UAV swarm combat scenarios is not feasible. Recently, artificial intelligence-based methods have been widely used in UAV swarm confrontation, including neural network methods [11, 20] and reinforcement learning methods [12–14, 21]. Neural network methods primarily involve using neural networks to encode maneuver rules, leading to substantial improvements in response speed and robustness. An adversarial method based on neural networks has been developed in [11], enhancing the autonomous decision-making abilities of intelligent agents. However, neural network training still necessitates a large volume of sample data. By integrating fuzzy neural networks and fractional

calculus in [20], the control performance of UAVs is significantly improved in the presence of actuator faults and model uncertainties, but implementing this control scheme requires substantial computing resources.

It's worth noting that reinforcement learning has emerged as a widely adopted approach for decision-making in UAV swarm confrontation [22]. This machine learning technique learns action policies by interacting with the environment, without the need for sample data [23]. Consequently, it proves to be an effective method for tackling sequential decision-making problems where prior models are lacking. In [12], the challenge of real-time maneuver decision-making in aerial combat has been addressed, which introduces a decision-making model based on reinforcement learning, incorporating multi-layer reward mechanisms and integrating them with the advantage actor-critic (A3C) algorithm. This approach significantly improves the UAV's maneuver decision-making capabilities in complex aerial combat scenarios. Utilizing long short-term memory (LSTM) networks and hindsight experience replay (HER) mechanisms in [21], the learning efficiency, convergence speed, and stability of the UAV algorithm are effectively enhanced, but it requires substantial computing resources and cannot be extended to complex environments. A collaborative decision-making algorithm based on multi-agent reinforcement learning has been presented in [13], which utilizes the actor-critic algorithm with gated recurrent units to enable UAVs to incorporate historical information into the decision-making process. Moreover, it incorporates a central critic using an attention mechanism, resulting in improved learning outcomes in complex environments. Furthermore, a hierarchical multi-agent reinforcement learning (HMARL) algorithm has been proposed in [14], which designed three aerial combat sub-tasks and employs a combination of competitive self-play (SP) and recursive soft actor-critic (RSAC) techniques to learn sub-policies for high-level goal selection and low-level policy execution. This hierarchical approach is shown to effectively handle adversarial situations involving variable-scale formations.

The proximal policy optimization (PPO) algorithm has emerged as a notable reinforcement learning approach, demonstrating considerable advancements over previous algorithms like soft actor-critic (SAC) and asynchronous advantage actor-critic (A3C). PPO stands out due to its effective balancing of sample efficiency, stability, and simplicity, contributing to improved algorithm efficiency [24]. In addition, the multi-agent PPO (MAPPO) algorithm has demonstrated distinctive advantages in addressing challenges related to multi-agent cooperation. By introducing more effective policy update mechanisms and adaptive reward designs, MAPPO enhances the performance of reinforcement learning systems, leading to significant improvements as highlighted in [25]. Additionally, the introduction of the Multi-Agent PPO (MAPPO) algorithm, has demonstrated unique advantages in solving multi-agent cooperation problems, especially showing promising results in UAVs confrontation scenarios in [26]. MAPPO further improves the performance of the reinforcement learning system by introducing more effective policy update mechanisms and adaptive reward designs [25].

In this article, we study the problem of UAV swarm confrontation and represent multi-UAV combat as a Markov game in continuous space. Then, a cooperative maneuver control method for UAVs is proposed, which employs the multi-agent proximal policy optimization (MAPPO) algorithm. The main contributions of this article are as follows.

- 1) This article presents a reinforcement learning modeling method that is suitable for both single-round and multi-round UAV swarm game confrontations. For single-round UAV game confrontations, based on the kinematic model, UAV poses information and survival information, we design the observation space and action space to construct a single-round UAV swarm game confrontation model. For multi-round UAV game confrontation, based on the number of our surviving UAVs and the number of enemy UAVs, determine the number of our UAVs to deploy. Moreover, considering the round information, a multi-round UAV swarm game confrontation model is also presented.
- 2) This article proposes a reward function scheme applicable to both single-round and multi-round UAV swarm game confrontations. The scheme incentivizes our UAVs to destroy more enemy UAVs while also guaranteeing their survival.
- 3) Moreover, by comparing with the proposed method in [26], simulation experiments are designed to evaluate the proposed algorithm from multiple aspects, such as enemy strategies, win rate, the number of evasions from enemy attacks, and the number of attacks on enemy UAVs, thus demonstrating the effectiveness of the proposed algorithm.

Notation.

- i, j represent natural numbers.
- $\dot{x}$ denotes the derivative of x .
- $\sin(\cdot)$ and $\cos(\cdot)$ denote the sine and cosine functions, respectively.
- $(\cdot)^\top$ denotes matrix transpose.
- $\|\cdot\|$ denotes the Euclidean norm.
- $\arccos(\cdot)$ denotes the inverse cosine function.
- Bold symbols represent vectors, for example, α .
- $\mathbb{N}$ represents the set of positive integers.
- $\mathbb{S}$ represents the state space matrix.
- $\mathbb{A}$ represents the joint action space matrix.
- $\mathbb{P}$ represents the state transition matrix.
- $\times$ denotes the vector cross product.
- $\odot$ denotes the Hadamard product (element-wise multiplication) for matrices.
- $\pi_\theta(a | b)$ represents the conditional probability.
- $\sum$ denotes summation.
- $\max(\cdot)$ denotes taking the maximum value.
- $\text{Clip}(\cdot)$ denotes clipping within a specified range.
- $T // L$ denotes the integer division of T by L .
- $\tau[a : b]$ denotes the time series from index a to b .
- $\hat{A}[a : b]$ denotes the action sequence from index a to b .
- $\hat{R}[a : b]$ denotes the reward sequence from index a to b .
- ∇ denotes the gradient or partial derivative.

2 | PRELIMINARIES AND PROBLEM FORMULATION

In this UAV swarm confrontation scenario, there are two teams: red and blue, with an equal number (n) of UAVs on each side. These UAVs have similar maneuverability and attack capabilities and contribute equally to their respective team's success. Both teams aim to destroy as many enemy UAVs as possible until all enemy UAVs are eliminated. It is crucial to devise an effective strategy that allows each UAV to make decisions based on the available information, enabling the UAV swarm to efficiently eliminate multiple targets in the shortest time possible. This requires each UAV to have access to information about both friendly and enemy UAVs.

This section will develop an adversarial environment for drone swarms, comprising the following four components:

- Drone kinematic model: Defining the kinematic characteristics and capabilities of the UAVs.
- Geometric formation model: Establishing the rules and principles governing the geometric formation of UAVs swarm.
- State and action space: Defining the state space and action space that the UAV swarm will operate within.
- Adversarial tasks: Outlining two distinct levels of difficulty for the adversarial tasks, along with the corresponding win or lose criteria.

The goal is to create a comprehensive framework for evaluating the performance and decision-making capabilities of UAV swarms in challenging, adversarial scenarios.

2.1 | Drone kinematic model

The kinematic model of the UAVs for the red team is denoted by U_i^r for $i = 0, 1, \dots, n-1$. Similarly, the kinematic model of the UAVs for the blue team is denoted by U_j^b for $j = 0, 1, \dots, n-1$. In this study, it is assumed that all UAVs are flying at the same altitude, therefore their pitch and roll angles are both 0 and the heading angle θ represents the angle between their velocity direction and the x -axis of the ground coordinate system. The maneuverability of each UAV is controlled in real-time by two parameters: velocity v and change in heading angle ω . Thus, the kinematic model for both sides' UAVs can be represented as:

$$\begin{cases} \dot{x}_i &= v_i \cos \theta_i, \\ \dot{y}_i &= v_i \sin \theta_i, \\ \dot{\theta}_i &= \omega_i, \end{cases} \quad i = 0, 1, \dots, n-1, \quad (1)$$

$$\begin{cases} \dot{x}_j &= v_j \cos \theta_j, \\ \dot{y}_j &= v_j \sin \theta_j, \\ \dot{\theta}_j &= \omega_j, \end{cases} \quad j = 0, 1, \dots, n-1. \quad (2)$$

Here, $P_i = (x_i, y_i)^\top$ and $P_j = (x_j, y_j)^\top$ represent the positions of U_i^r and U_j^b , respectively, where v_i and v_j denote the

speeds of U_i^r and U_j^b , respectively. θ_i and θ_j represent the heading angles of U_i^r and U_j^b , respectively, while ω_i and ω_j denote the rates of change of heading angles for U_i^r and U_j^b , respectively. Moreover, $v_i = (v_i \cos(\theta_i), v_i \sin(\theta_i))^T$ and $v_j = (v_j \cos(\theta_j), v_j \sin(\theta_j))^T$ represent the velocity vectors of U_i^r and U_j^b , respectively.

In this article, unless otherwise specified, the subscript i denotes quantities related to the UAV U_i^r for the red team, while the subscript j denotes quantities related to the UAV U_j^b for the blue team.

2.2 | Geometric formation model

In the scenario of a red-blue UAV swarm confrontation, it is assumed that there are n UAVs on each team. At time t , taking U_i^r and U_j^b as examples, for the red team's drone U_i^r , its attack capability is described by the effective attack area, defined as a red sector with itself as the center, velocity v_i direction as the central line, a central angle of $2\phi_i$, and radius of ρ_i ; if an opposing UAV enters this sector, it will be considered destroyed. The effective attack range of the blue team's UAV U_j^b is similar.

For the red team's UAV U_i^r , the distance vector to target UAV U_j^b , is defined as $\mathbf{d}_{ij} = P_i P_j$, where the distance d_{ij} is calculated as:

$$d_{ij} = \|\mathbf{d}_{ij}\| = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}. \quad (3)$$

The target bearing angle, denoted by α_{ij} , is defined as the angle between the current velocity direction of the red team's UAV and $\mathbf{d}_{ij}$, which can be given as follows.

$$\alpha_{ij} = \arccos\left(\frac{\mathbf{d}_{ij} \cdot \mathbf{v}_i}{\|\mathbf{d}_{ij}\| \cdot \|\mathbf{v}_i\|}\right), \quad 0 \leq \alpha_{ij} \leq \pi. \quad (4)$$

Moreover, the angle between the velocity direction of target U_j^b and $\mathbf{d}_{ij}$ is defined as the target entry angle, denoted by β_{ij} , which can be given by:

$$\beta_{ij} = \arccos\left(\frac{\mathbf{d}_{ij} \cdot \mathbf{v}_j}{\|\mathbf{d}_{ij}\| \cdot \|\mathbf{v}_j\|}\right), \quad 0 \leq \beta_{ij} \leq \pi. \quad (5)$$

Similarly, for the blue team's UAV U_j^b , the distance to target UAV U_i^r is given as $d_{ji} = d_{ij}$, the target bearing angle is $\alpha_{ji} = \pi - \beta_{ij}$, and the target entry angle can be obtained as $\beta_{ji} = \pi - \alpha_{ij}$.

Hence, according to the given geometric parameters, the current 1 vs 1 confrontation situation can be categorized into the following four cases, shown in Figure 1.

- The red team has the advantage when $\alpha_{ij} < \pi/2$ and $\beta_{ij} < \pi/2$, indicating that the blue team is positioned in front of the red team and moving away from them.

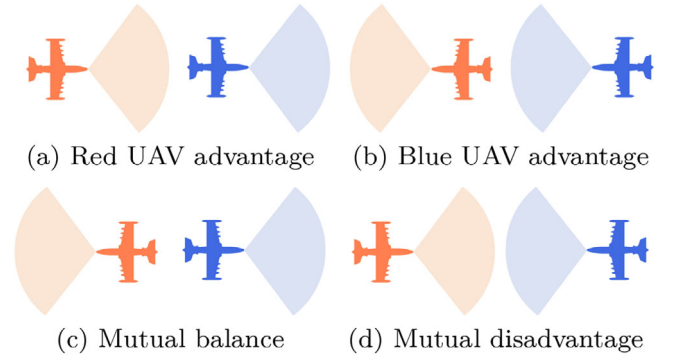


FIGURE 1 The confrontation situation of a red UAV and a blue UAV.

- The blue team has the advantage when $\alpha_{ij} > \pi/2$ and $\beta_{ij} > \pi/2$, indicating that the red team is located in front of the blue team and moving away from the blue team.
- In a balanced situation, when $\alpha_{ij} \geq \pi/2$ and $\beta_{ij} \leq \pi/2$, it indicates that both sides are positioned on each other's rear and moving away from each other, or moving in the same direction.
- When both $\alpha_{ij} < \pi/2$ and $\beta_{ij} > \pi/2$, it suggests that both sides are at a disadvantage, positioned in front of each other.

In a confrontation between n UAVs from the red team and n drones from the blue team, each UAV must consider not only the geometric parameters between opposing UAVs but also those among its own team's UAVs. For the red team's UAV U_i^r , the distance vector to its teammate UAV U_{-i}^r ($-i = 0, \dots, i-1, i+1, \dots, n-1$) is defined as $\mathbf{d}_{i,-i}$, with the target bearing angle and the target entry angle denoted by $\alpha_{i,-i}$ and $\beta_{i,-i}$, respectively. Similarly, for the blue team's UAV U_j^b , the distance vector to its teammate UAV U_{-j}^b ($-j = 0, \dots, j-1, j+1, \dots, n-1$) is defined as $\mathbf{d}_{j,-j}$, with the target bearing angle and the target entry angle denoted by $\alpha_{j,-j}$ and $\beta_{j,-j}$, respectively.

Based on the geometric relationships between the red and blue drones, it is possible to determine if a specific drone is within the effective attack range of an opposing drone, thus establishing the survival status of that drone. For the red team's UAV U_i^r , its survival status is represented by a logical variable $L_i^r \in \{0, 1\}$, where $L_i^r = 0$ indicates destruction and $L_i^r = 1$ indicates survival. Therefore, the survival status of the red team's UAVs is denoted as $\mathbf{L}^r = (L_0^r, \dots, L_{n-1}^r)$, and that of the blue team's UAVs is denoted as $\mathbf{L}^b = (L_0^b, \dots, L_{n-1}^b)$. In the confrontation between the red and blue teams, for example, considering the blue team's UAV U_j^b , its initial survival status value is 1, and the update rule is as follows:

$$L_j^b = \begin{cases} 0, & \text{if } \exists i \in \{0, 1, \dots, n-1\}, \text{ s.t. } \alpha_{ij} \leq \phi_i \text{ and } d_{ij} \leq \rho_i, \\ L_j^b, & \text{otherwise.} \end{cases} \quad (6)$$

Therefore, by continuously evaluating the geometric relationships and survival statuses of the red and blue teams' UAVs,

it becomes feasible to strategize and optimize the overall team performance. This comprehensive analysis allows for informed decision-making regarding defensive and offensive maneuvers, ultimately contributing to the overall success of the team's mission objectives.

2.3 | State and action space

In the simulation environment of UAV swarm confrontation, it is assumed that each UAV shares its own and the enemy UAVs' relative position information and survival status. Taking the red team's UAV U_i^r as an example, the distance and angle relationships between U_i^r and its neighboring teammate UAVs U_{-i}^r are represented by the sets $\mathbf{d}_i = \{d_{i,-i}\}$, $\boldsymbol{\alpha}_i = \{\alpha_{i,-i}\}$, and $\boldsymbol{\beta}_i = \{\beta_{i,-i}\}$, where $-i = 0, \dots, i-1, i+1, \dots, n-1$. Similarly, the distance and angle relationships with the opponent blue team's UAV U_j^b are represented by the sets $\bar{\mathbf{d}}_i = \{d_{ij}\}$, $\bar{\boldsymbol{\alpha}}_i = \{\alpha_{ij}\}$, and $\bar{\boldsymbol{\beta}}_i = \{\beta_{ij}\}$, where $j = 0, 1, \dots, n-1$. The number of rounds in the multi-round confrontation is represented by F . Collectively, these geometric relationships constitute the observation space of the red team's UAV U_i^r , which can be given as:

$$\mathbf{d}_i^r = [\mathbf{d}_i, \boldsymbol{\alpha}_i, \boldsymbol{\beta}_i, \bar{\mathbf{d}}_i, \bar{\boldsymbol{\alpha}}_i, \bar{\boldsymbol{\beta}}_i, L^r, L^b, F]. \quad (7)$$

As the maneuverability of all UAVs U_i is dynamically controlled using two continuous parameters, namely velocity v_i and change in heading angle ω_i , the action space of U_i^r is defined as $\mathbf{a}_i^r = [v_i^r, \omega_i^r]$. The observation space $\mathbf{d}_i^b$ and action space $\mathbf{a}_i^b$ of the blue team's UAVs are similarly defined using this approach, ensuring uniform representation and control across the simulation environment.

2.4 | Adversarial tasks

Taking into account that each drone is equipped with identical maneuverability and attack capabilities, and holds an equal influence on the overall outcome, the subsequent training is designed around the following confrontation task. Within both the red and blue teams, each drone is considered to hold an equivalent rank, signifying that the survival or destruction of any individual drone equally impacts the team's overall success or failure. Therefore, the team's objective is to indiscriminately destroy a greater number of enemy drones. After the single-round confrontation, victory or defeat is determined by assessing the survival states L^r and L^b of both sides' drones. The side that successfully shoots down more enemy drones emerges as the victor; in the event of an equal number of shootdowns, the outcome is a draw. The winning scenarios for the blue team are the opposite of those for the red team, and any outcome other than victories for both the red and blue teams is considered a draw.

For multi-round confrontations, the first round commences with a 3v3 confrontation between the red and blue teams. At this stage, all of the blue team's UAVs have been destroyed.

Before the start of the second-round confrontation, the survival status L of both teams' UAVs will be used as a determining factor: if the red team's survival state L^r is not equal to zero but the blue team's survival state L^b is zero, the blue team will continue to dispatch three UAVs to engage in combat with the remaining red UAVs. This process persists until all red UAVs are eliminated or the maximum number of rounds is reached.

The confrontation task, characterized by its symmetric yet opposing winning conditions for the red and blue teams, conforms to the framework of a Markov game. In a Markov game, the outcomes of each player's actions are contingent not only on the current state of the game but also on the actions taken by all players. The structure of this game, where the actions and outcomes of each team's drones have a collective impact on overall success, embodies the defining attributes of a Markov game. The strategic interplay between the red and blue teams, with their symmetric yet opposing objectives, underscores the intricate and interdependent nature of the game, rendering it suitable for the analysis within the framework of a Markov game in the next subsection.

2.5 | Markov games

According to the aforementioned adversarial scenario and the rules for determining victory or defeat, each side in the conflict between the red and blue UAV swarms can be regarded as a fully cooperative multi-agent system. We have made further improvements based on MAPPO in [26]. Therefore, this article models their decision-making process as a Markov Game, represented by the tuple $\langle \mathbb{N}, \mathbb{S}, \mathbb{A}^i, \mathbb{P}, R^i, \gamma \rangle$.

In this tuple, $\mathbb{N} = \{1, \dots, n\}$ represents a finite set of $n \geq 2$ intelligent agents. $\mathbb{S}$ denotes the actual state-space of the environment, while the joint action space is represented by $\mathbb{A}^i = \mathbb{A}^1 \times \dots \times \mathbb{A}^n$ (where $i \in \mathbb{N}$), with each $\mathbb{A}_i$ being the action space set of all n UAVs U_i within our swarm. The transition matrix $\mathbb{P} : \mathbb{S} \times \mathbb{A} \rightarrow \mathbb{P}(\mathbb{S})$ describes the probability of state transitions in the multi-agent system. Each intelligent agent has a corresponding reward function $R^i : \mathbb{S} \times \mathbb{A} \times \mathbb{S} \rightarrow \mathbb{R}$, which provides immediate feedback signals. The parameter $\gamma \in [0, 1)$ represents the discount factor. In the red-blue UAV swarm confrontation, the state-space $\mathbb{S}$ of the UAVs for both the red and blue teams include the position (x_i, y_i) , velocity v_i , heading angle θ_i , survival status L_i^r and L_i^b and the number of rounds in the multiple-round confrontation F . The action space $\mathbb{A}^i$ is composed of velocity v_i and change in heading angle ω_i . The state transition function $\mathbb{P}$ describes the probability of transitioning from the current state to the next state given specific actions. The reward function R^i is determined based on the situation of enemy UAVs being shot down, and each UAV maximizes its cumulative reward by choosing its policy π^i .

Specifically, at time t , each UAV U_i within our swarm interacts with the environment based on its own state s_t . Based on its own strategy $\pi_i : \mathbb{S} \rightarrow P(\mathbb{A}^i)$, it selects and executes an action a_t^i . Then, the environment transitions from state $s_t \in \mathbb{S}$ to the next state $s_{t+1} \in \mathbb{S}$ based on the joint actions $a_t = (a_t^1, \dots, a_t^n)$ of all UAVs within our swarm according to the transition matrix

$\mathbb{P}$. At the same time, each UAV within our swarm receives an immediate reward r_t^i from the environment as feedback. Each UAV within our swarm aims to find the optimal strategy π^* to optimize the cumulative reward from a long-term perspective. Since the UAVs within our swarm are fully cooperative, each UAV has the same reward and objective functions. The training aims to maximize the objective function

$$J(\pi) = \mathbb{E}_{s_0, a_0} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right]. \quad (8)$$

$J(\pi)$ is the total return of policy π . $\mathbb{E}_{s_0, a_0}$ denotes the expectation over the initial state and action. $\sum_{t=0}^{\infty}$ denotes the cumulative sum from time step $t = 0$ to infinity. γ^t is the discount factor, $\gamma \in [0, 1]$, used to discount future rewards. A smaller discount factor means that future rewards have less impact on current decisions. $R(s_t, a_t)$ is the immediate reward obtained by taking action a_t in the state s_t . The objective function defined in Equation (8) is used to define the objective function for training UAVs. It represents the expected return $J(\pi)$ of a UAV's policy π , which is the expected cumulative discounted reward of all UAVs from time step t from 0 to infinity.

3 | DECISION-MAKING FRAMEWORK

In this section, we propose a methodological framework for addressing the challenge of UAV swarm confrontation. For the proposed Markov game model in Section 2.5, the MAPPO algorithm is employed for iterative policy optimization. A new reward function scheme for UAV swarm confrontation is introduced, based on an established adversarial gaming environment model, and features a unique representation for the reward function.

3.1 | Multi-agents proximal policy optimization

By extending the capabilities of the proximal policy optimization (PPO) algorithm, MAPPO is tailored to address the intricacies stemming from cooperation and competition within multi-agent environments. In multi-agent scenarios, each agent possesses its own observation space, action space, and policy. MAPPO applies the PPO algorithm to the policy network of each agent, considering the actions of other agents, to enable collaborative decision-making across the entire system. The primary objective is to train adaptive and coordinated behavior policies for each agent through policy network training, ultimately striving for global optimality, we have made further improvements based on MAPPO in [26].

For the Markov game process established in Section 2.5, the MAPPO algorithm is used for training. The training process begins by initializing the scenario, the number of agents, and the network parameters. In each training cycle, the states, actions, and reward data of the agents are collected through batch

processing. The advantage values are then calculated using generalized advantage estimation (GAE). The trajectories are split into smaller chunks. For each chunk, a GRU (gated recurrent unit) is used to update the hidden states of the policy network and the value network. Finally, the network parameters are updated using the Adam optimizer. This iterative process continues until the maximum number of training steps is reached, allowing the multi-agent system's strategy to be optimized.

During the training phase, the MAPPO algorithm follows a strategy where the blue-side approach is established first, and only the red-side UAVs undergo training. The algorithm employs centralized training and distributed execution, establishing two separate neural networks for training: an actor-network with parameters θ , π_θ , and a critic network with parameters ϕ , V_ϕ . Since all red-side UAVs share a similar observation space and action space, they also share the parameters of the actor and critic networks. The critic network plays a role in aiding intelligent training but is not involved in centralized observation input. After training, each UAV utilizes its local observations o_i^k to independently make decisions using the actor network, without requiring centralized observation input. To ensure continuity in the UAV action space, the output of the actor network is structured as a multi-dimensional Gaussian distribution with mean and standard deviation, and an action a_i^t is obtained using a sampling method. Additionally, the critic network takes the global observation s_t as input and produces the estimated value function of the state value function. This process is depicted in Figure 2.

Training the actor-network to maximize the objective function:

$$L(\theta) = \left[\frac{1}{Bn} \sum_{i=1}^B \sum_{k=1}^n \min \left(r_{\theta,i}^{(k)} A_i^{(k)}, f \left(r_{\theta,i}^{(k)} \right) \right) \right] + \sigma \frac{1}{Bn} \sum_{i=1}^B \sum_{k=1}^n \mathcal{S} \left[\pi_\theta \left(o_i^{(k)} \right) \right], \quad (9)$$

where

$$f(r_{\theta,i}^{(k)}) = \text{clip} \left(r_{\theta,i}^{(k)}, 1 - \epsilon, 1 + \epsilon \right) A_i^{(k)},$$

$$r_{\theta,i}^{(k)} = \frac{\pi_\theta \left(a_i^{(k)} | o_i^{(k)} \right)}{\pi_{old} \left(a_i^{(k)} | o_i^{(k)} \right)}.$$

Here, $A_i^{(k)}$ is the advantage function, calculated using the generalized advantage estimation method, $\mathcal{S}$ represents the policy entropy, σ is the entropy coefficient hyperparameter, and ϵ is the hyperparameter to constrain the update range of the policy. Training the critic network to minimize the loss function:

$$L(\phi) = \frac{1}{Bn} \sum_{i=1}^B \sum_{k=1}^n \left[\max \left(\Omega_1, (\Omega_2 - \hat{R}_i) \right)^2 \right], \quad (10)$$

ALGORITHM 1 Recurrent-MAPPO.

Input: Types of scenarios env_{name} and the number of parallels env_{num} , our agents' number, algorithms' type, the number and strategy of the enemy, total training steps $step_{max}$, and step length per round T . Initialize θ , the parameters for policy π ; initialize ϕ , the parameters for critic V ; Set learning rate α .

Output: Actor-network and Critic-network, Optimal strategy π^* .

```

1 while  $step \leq step_{max} / (T \times env_{num})$  do
2   set  $D_{buffer} = []$ ;
3   for  $i = 1$  to  $batch_{size}$  do
4      $\tau = []$ ;
5     initialize actor GRU  $h_{0,\pi}^{(1)}, \dots, h_{0,\pi}^{(n)}$ ;
6     initialize critic GRU  $h_{0,V}^{(1)}, \dots, h_{0,V}^{(n)}$ ;
7     for  $t = 1$  to  $T$  do
8       for for all agent  $a$  do
9          $p_t^{(a)}, h_{t,\pi}^{(a)} = \pi(o_t^{(a)}, h_{t-1,\pi}^{(a)}; \theta)$ ;
10         $u_t^{(a)} \sim p_t^{(a)}$ ;
11         $v_t^{(a)}, h_{t,V}^{(a)} = V(s_t^{(a)}, h_{t-1,V}^{(a)}; \phi)$ ;
12      end
13      Execute actions  $u_t$ , observe  $r_t, s_{t+1}, o_{t+1}$ ;
14       $\tau = [s_t, o_t, h_{t,\pi}, h_{t,V}, u_t, r_t, s_{t+1}, o_{t+1}]$ ;
15    end
16    Compute advantage estimate  $\hat{A}$  via Generalized Advantage Estimation (GAE) on  $\tau$ ;
17    Compute reward-to-go  $\hat{R}$  on  $\tau$  and normalize with Popart;
18    Split trajectory  $\tau$  into chunks of length  $L$ ;
19    for  $l = 0, 1, \dots, T/L$  do
20       $D = D \cup (\tau[l : l + T], \hat{A}[l : l + T], \hat{R}[l : l + T])$ ;
21    end
22  end
23  for  $mini_{batch} \ k = 1, \dots, K$  do
24     $b \leftarrow$  random mini-batch from  $D$  with all agent
25    datafor each data chunk  $c$  in the  $mini_{batch} \ b$  do
26      update GRU hidden states for  $\pi$  and  $V$  from first hidden state in data chunk;
27    end
28  end
29  Adam update  $\theta$  on  $L(\theta)$  with data  $b$ ;
30  Adam update  $\phi$  on  $L(\phi)$  with data  $b$ ;
31 end
32 return

```

round-based rewards on top of the single-round confrontation, while the penalty for failure remains unchanged. This encourages the UAV swarm to take the initiative to attack without being afraid of confrontation.

Innovation of this reward function lies in the incremental reward mechanism and the penalty design. It not only moti-

ALGORITHM 2 GRU.

Input: The features of agent $F_t^{(a)}$ and the hidden state $h_{t-1}^{(a)}$. Initialize gate parameters ω_z, u_z, b_z ; reset gate parameters ω_r, u_r, b_r ; candidate hidden state parameters ω_h, u_h, b_h ; output layer parameters ω_y, b_y ; and hidden state u_0 .

Output: The features of agent $OF_t^{(a)}$ and the hidden state $h_t^{(a)}$.

```

1 for  $t = 1$  to  $T$  do
2   Calculate update
3   gate  $z_t = \text{sigmoid}(\omega_z \times F_t^{(a)} + u_z \times h_{t-1}^{(a)} + b_z)$ , Calculate
4   update
5   gate  $r_t = \text{sigmoid}(\omega_r \times F_t^{(a)} + u_r \times h_{t-1}^{(a)} + b_r)$ , Calculate
6   the candidate's hidden state  $h_t^{hat} = \tanh(\omega_h \times F_t^{(a)} + u_h \times (r_t \odot h_{t-1}^{(a)}) + b_h)$ , Calculate
7   output hidden
8   state  $h_t^{(a)} = (1 - z_t) \odot h_{t-1}^{(a)} + z_t \odot h_t^{hat}$ , Calculate output
9   features  $OF_t^{(a)} = \text{softmax}(\omega_y \times h_t^{(a)} + b_y)$ .
10 end
11 return

```

vates the UAV swarm to maintain efficient offensiveness in the later stages of combat but also ensures, through the penalty mechanism, that the UAV swarm minimizes early losses, maintaining team coordination and defensive awareness. This makes UAVs more diverse in complex adversarial scenarios, providing an efficient strategy-guiding mechanism.

The env_{num} environment corresponds to a 3vs3 UAV confrontation scenario, with specific parameters detailed in Section 4. These parameters should be configured according to the specific environment. A gated recurrent unit (GRU) is employed to extract the features of observation information and hidden states. The main process of the GRU is outlined in Algorithm 2. To expedite the training process and enhance stability, the Adaptive Moment Estimation (ADAM) optimizer is utilized to adjust θ_t . The main process of ADAM is illustrated in Algorithm 3.

4 | EXPERIMENTS

4.1 | Experimental parameter settings

This article utilizes Python to construct an adversarial testing platform for UAV swarm scenarios. The training framework for the MAPPO algorithm is developed on CUDA 11.1 and Torch 1.8.1. Comprehensive details regarding the relevant software and hardware parameters of the experimental platform are presented in Table 1, while algorithm hyperparameters are outlined in Table 2.

The total computational complexity of the algorithm during training is:

ALGORITHM 3 ADAM.

Input: Stepsize α_1 , Exponential decay rates for the moment estimates $\beta_1, \beta_2 \in [0, 1)$, Stochastic objective function $L(\theta)$. Initialize the parameter vector θ_0 , exponential moving averages of the gradient m_0 , squared gradient v_0 , the timestep t .

Output: Resulting parameters θ_t .

```

1 while  $\psi_t$  not converged do
2    $t \leftarrow t + 1$ ;
3   Get gradients objective:  $g_t \leftarrow \nabla_{\psi} f_t(\phi_{t-1})$ ;
4   Update biased first moment estimate:
      $m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ ;
5   Update biased second raw moment estimate:
      $v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ ;
6   Compute bias-corrected first moment
     estimate:  $\hat{m}_t \leftarrow \frac{m_t}{1 - \beta_1^t}$ ;
7   Compute bias-corrected second raw moment
     estimate:  $\hat{v}_t \leftarrow \frac{v_t}{1 - \beta_2^t}$ ;
8   Update parameters:  $\theta_t \leftarrow \theta_{t-1} - \frac{\alpha \cdot \hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$ .
9 end
10 return

```

TABLE 1 Parameters of the experimental platform.

	Parameter	Value
Software	Simulation environment	Python 3.6.2
	Operating system	Ubuntu 22.04
Hardware	CPU	Inter i9-13900HX
	GPU	NVIDIA RTX 4080
	RAM	32G

TABLE 2 The hyperparameters of MAPPO algorithm.

Parameter	Value
Actor learning rate	0.0005
Critic learning rate	0.0005
PPO epoch	10
Clip param	0.1
Entropy coef	0.01
GAE lambda	0.95
Gamma	0.99
Episode length	150

$$O(E \times M \times L \times (N_{\text{friend}} \times (N_{\text{friend}} + N_{\text{enemy}} - 1) + 2 \times (d_{\text{input}} \times b + b^2) + 3 \times b^2) + P),$$

where the parameters in the formula are defined as follows: E is the number of parallel environments used during train-

TABLE 3 Red team UAVs performance parameters setting.

Parameter	Value
ρ_i (km)	5
φ_i (rad)	$\pi/4$
v_i (m/s)	[120, 600]
ω_i (rad/s)	$[-\pi/12, \pi/12]$

ing; M is the number of training rounds for each environment; L represents the number of training steps per round; N_{friend} and N_{enemy} are the numbers of surviving friendly and enemy UAVs, respectively; d_{input} is the input feature dimension; b is the number of neurons in the hidden layers of the neural network. The term $2 \times (d_{\text{input}} \times b + b^2)$ represents the computational complexity of the two fully connected layers, where $d_{\text{input}} \times b$ corresponds to the first layer and b^2 refers to the hidden-to-hidden connections. Additionally, $3 \times b^2$ accounts for the GRU layer complexity, which involves three gates, while $N_{\text{friend}} \times (N_{\text{friend}} + N_{\text{enemy}} - 1)$ describes the computational complexity of calculating the position and orientation relationships between the surviving friendly and enemy UAVs; P represents the computational complexity of updating the neural network parameters.

During the training, the initial positions of both red and blue team UAVs are randomly generated within the range of $x \times y \in (-50, 50 \text{ km}) \times (-50, 50 \text{ km})$. The initial heading angles for red team UAVs are randomly sampled from the interval $[0, \pi/2]$, while for blue team UAVs, they are randomly sampled from the interval $[-\pi/2, -\pi]$. The simulation sampling interval is set to 1 s. The parameters for the red team UAVs are provided in Table 3.

4.2 | Experiment and result analysis

The experiment involves a 3vs3 adversarial gameplay between red and blue team UAVs. During testing:

- The initial positions of the red team's UAVs are randomly generated within the range of $x \times y \in (-10, -50 \text{ km}) \times (-10, -50 \text{ km})$.
- The initial positions of the blue team's UAVs are randomly generated within the range of $x \times y \in (10, 50 \text{ km}) \times (10, 50 \text{ km})$.
- The red team's UAVs are controlled using the MAPPO algorithm.
- The blue team's UAVs use the Hungarian algorithm to allocate the positions of the red team's UAVs as target points.
- The blue team's UAVs periodically update their target points based on the current positions of the red team's UAVs, adjusting their trajectories accordingly.

A UAV is considered destroyed when its central area enters the attack range of the enemy. The adversarial gameplay round ends if one of the following conditions is met:

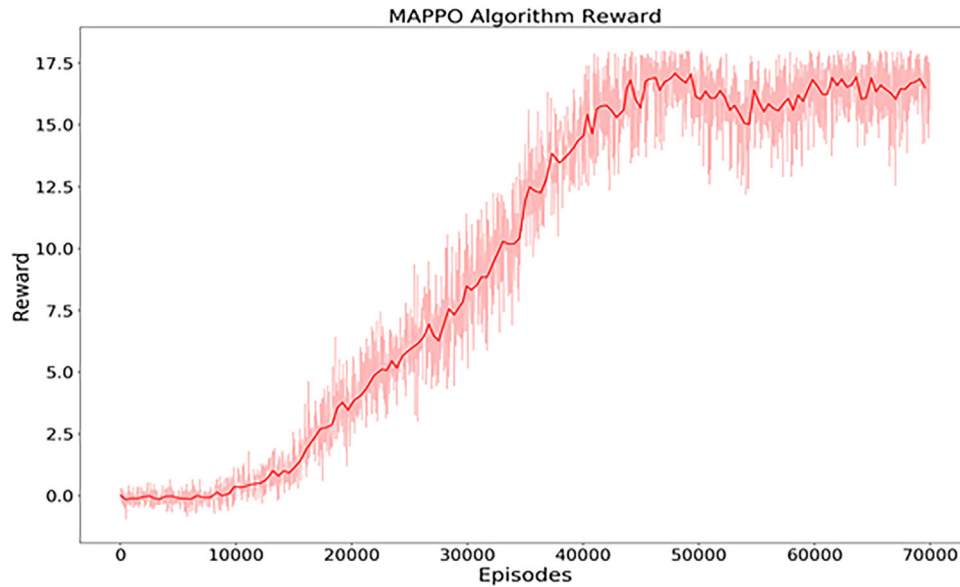


FIGURE 3 Average reward convergence figure in scenario 1.

1. The episode length is reached.
2. All UAVs within the red or blue team are destroyed.

Scenario 1: The blue team's UAVs utilize the positions of the red team's UAVs as target points. Every four steps, the target point positions are updated, prompting the blue team's UAVs to adjust their heading angles based on the new target point positions. Additionally, the speed is set at 400 m/s for the UAVs.

Scenario 2: The blue team's UAVs utilize the positions of the red team's UAVs as target points. Every two steps, the target point positions are updated, prompting the blue team's UAVs to adjust their heading angles based on the new target point positions. Additionally, the speed is set at 500 m/s for the UAVs.

The average reward curve for the red team's UAVs trained using the MAPPO algorithm is depicted in Figure 3. In the initial 10,000 rounds of training, the red team's UAVs explore strategies with only a slight increase in reward. However, as the number of adversarial rounds increases, the red team's UAVs gradually discover some strategies to defeat the blue team's UAVs, leading to a gradual increase in average reward. By the time training reaches 50,000 rounds, the reward begins to converge, indicating that the red team's UAVs have learned effective strategies to defeat the blue team's UAVs. To demonstrate the effectiveness of the proposed algorithm, we adopt the reward function setting method from reference [26] for comparison. The comparison results are shown in Figure 4, where the blue curve represents the reward convergence results from reference [26], and the red curve represents the convergence results of the reward function proposed in this article. As shown, although the proposed reward function has a slower initial growth, it achieves better convergence in the later stages.

For Scenario 1, we conduct 50 sets of UAV swarm confrontation tests, the win rate and average reward are shown in Table 4. The test results indicate that in these 50 tests, using the algorithm proposed in this article to control the UAVs, there is only

TABLE 4 Win rate and average reward in scenario 1.

	Win	Failure	Draw	Average reward
Tests 1 to 10	90%	10%	0%	15
Tests 11 to 20	100%	0%	0%	15.9
Tests 21 to 30	100%	0%	0%	16.2
Tests 31 to 40	100%	0%	0%	16.8
Tests 41 to 50	100%	0%	0%	16.8

one instance of failure in the confrontation tests. The remaining 49 tests are successful, resulting in an overall win rate of 98%.

Figures 5–9 show the maneuvering trajectories and real-time situational map of both the red and blue team's UAVs during Scenario 1 testing.

As illustrated in Figures 5–9, during the initial phase of the confrontation, the three red UAVs employ a cooperative encirclement tactic. They prioritize targeting the nearest blue UAV, designated as blue plane 2. While red plane 3, positioned farthest from the battlefield's center, adjusts its heading angle and advances directly toward the targeted blue plane 2, red planes 1 and 2 capitalize on their proximity to each other to synchronize their movements. Red plane 1 provides cover for red plane 2, enabling the latter to maneuver around the flank of the targeted blue plane 2 and gain a positional advantage. At $t = 58$ s, red plane 2 successfully eliminates the targeted blue plane 2.

Following these initial encounters, the red plane 3 swiftly enters the fray. To avoid a confrontation with Blue Plane 3, it maneuvers actively to change direction, drawing blue plane 3 into pursuit. Meanwhile, red plane 2 employs a deceptive rotational maneuver to mislead blue plane 1 into an attack, while also adjusting its position to prepare for an assault on blue plane 3, which is pursuing red plane 3. At the same time, red plane 1 positions itself alongside blue plane 1 and successfully

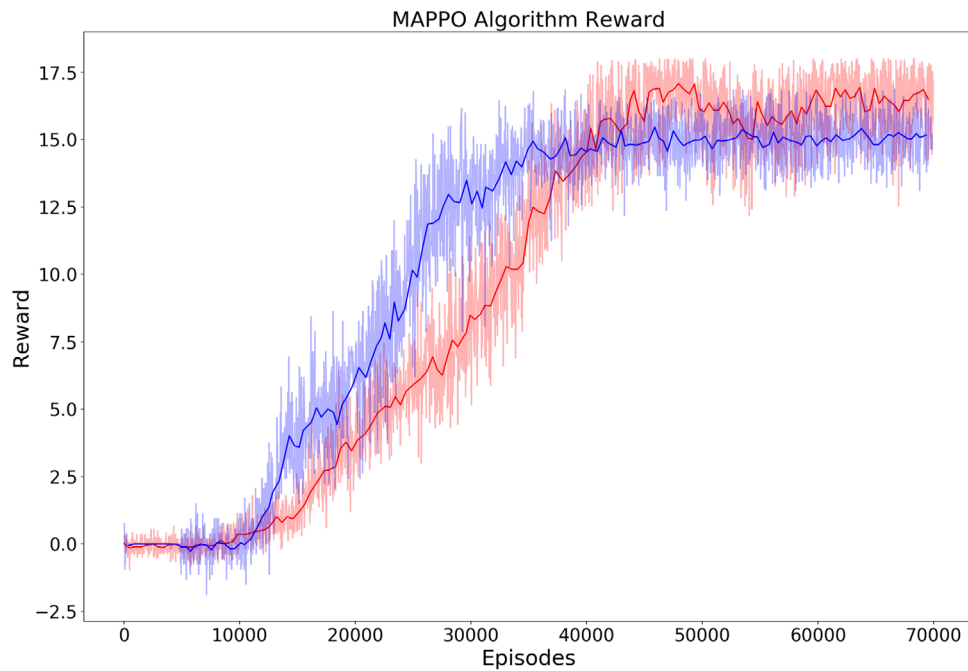


FIGURE 4 Convergence results of different reward functions in scenario 1.

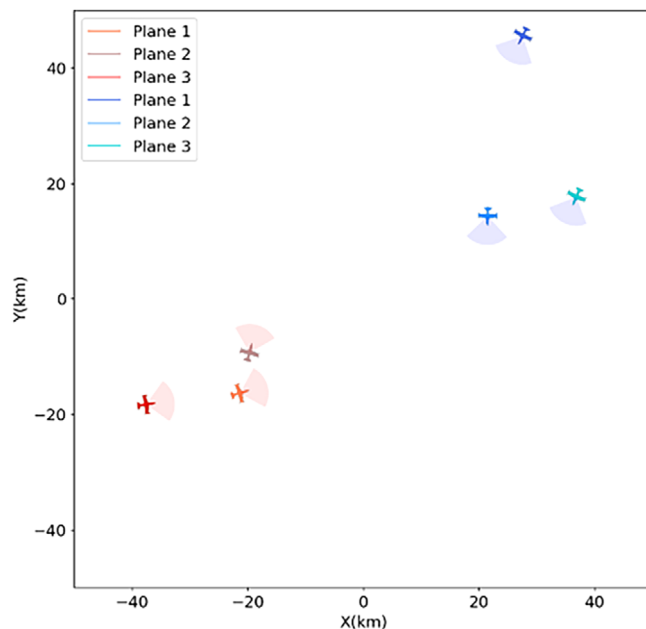


FIGURE 5 Situational of UAVs at the initial moment.

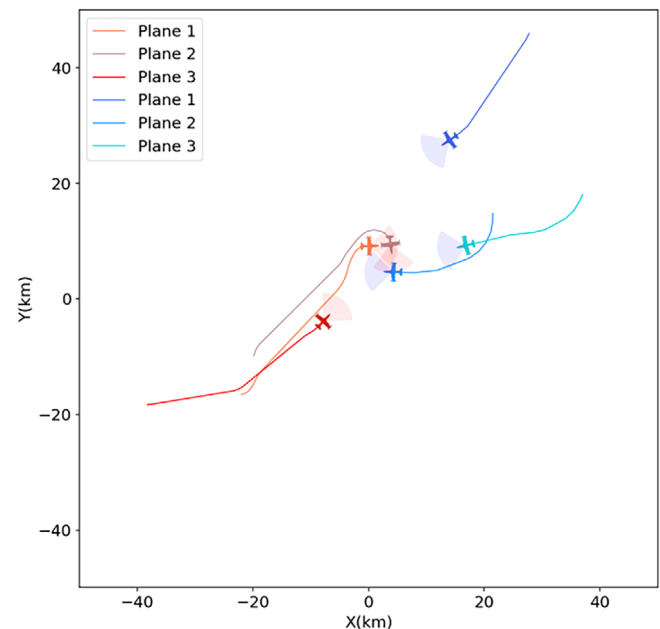


FIGURE 6 Maneuvering trajectories and situational at 58 s.

eliminates it at $t = 85$ s. Subsequently, at $t = 96$ s, red plane 2 maneuvers behind blue plane 3 and eliminates it. As a result, the red team's UAVs achieved victory by successfully eliminating all of the blue team's UAVs in the confrontation.

In Scenario 2, the MAPPO algorithm is used to train the red team's UAVs again, this time using the model from Scenario 1. The average reward curve for the red team shows some interesting dynamics:

- In the initial stages of training, the strategies previously explored by the red team's UAVs become less effective due to changes in the blue team's tactics. This results in a decrease in average reward, dropping to around half of the original value.
- However, as the training progresses and the number of rounds increases, the red team's UAVs can discover new strategies to defeat the blue team's UAVs. This leads to a gradual increase in the average reward.
- By the time the training reaches 20,000 rounds, the reward starts to converge, indicating that the red team's UAVs have

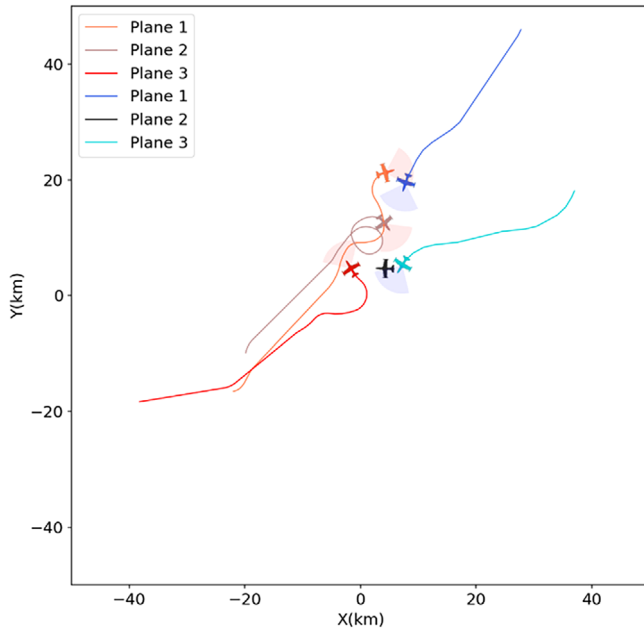


FIGURE 7 Maneuvering trajectories and situational at 85 s.

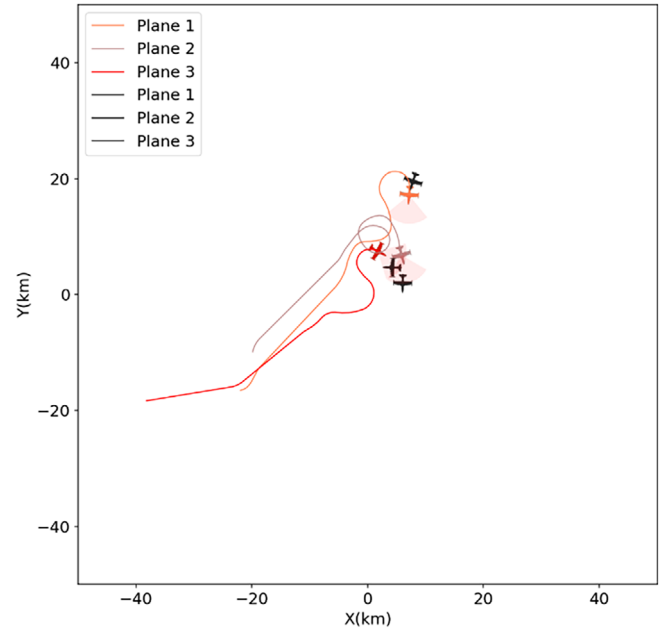


FIGURE 9 UAV adversarial outcome.

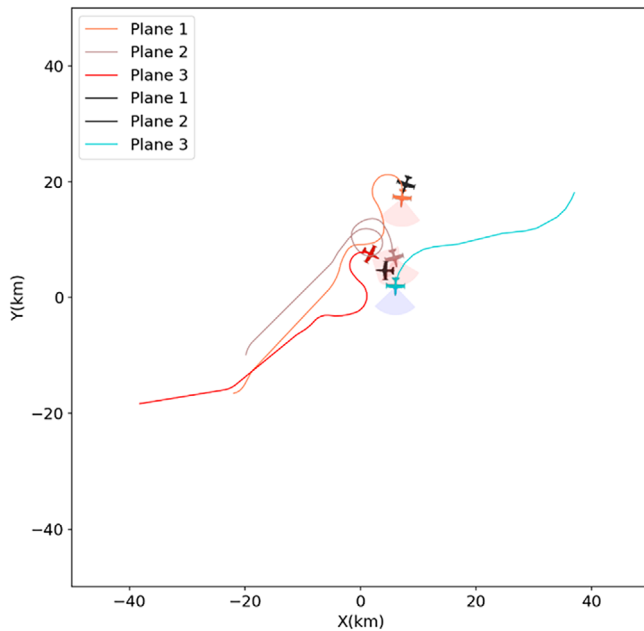


FIGURE 8 Maneuvering trajectories and situational at 96 s.

successfully learned new strategies to overcome the blue team's updated tactics.

This adaptive learning process demonstrates the flexibility and resilience of the MAPPO algorithm in adjusting the red team's tactics to counter the evolving strategies of the blue team's UAVs. The convergence of the reward curve suggests that the red team has developed an effective set of maneuvers and tactics to achieve victory in this new scenario.

For Scenario 2, the average reward curve for the red team's UAVs trained using the MAPPO algorithm is depicted in Figure

TABLE 5 Win rate and average reward in scenario 2.

	Win	Failure	Draw	Average reward
Tests 1 to 10	80%	10%	10%	12.9
Tests 11 to 20	80%	10%	10%	13.2
Tests 21 to 30	90%	10%	0%	14.1
Tests 31 to 40	90%	0%	10%	15
Tests 41 to 50	100%	0%	0%	15.6

10. Moreover, we conduct 50 sets of UAV swarm confrontation tests, the win rate and average reward are shown in Table 5. The test results indicate that in these 50 tests, using the algorithm proposed in this article to control the UAVs, there are three instances of failure and 3 draws in the confrontation tests. The remaining 44 tests are successful, resulting in an overall win rate of 88%.

Figures 11–15 show the maneuvering trajectories and real-time situational map of both the red and blue team's UAVs during Scenario 2 testing.

In the depicted scenario (Figures 11–15), the initial phase of the confrontation witnesses the three red planes employing a coordinated encirclement strategy, targeting the nearest blue plane, denoted as blue plane 3. Positioned farthest from the battlefield's center, red plane 3 adjusts its trajectory and heads directly towards blue plane 3. Capitalizing on their initial proximity, red planes 1 and 2 synchronize their movements. As red plane 2 deceives blue plane 3 with a change in direction, red plane 1 maneuvers to gain a positional advantage alongside blue plane 3. At $t = 65$ s, red plane 1 successfully eliminates blue plane 3. The coordination between red planes 1 and 2 continues, with red plane 1 deceiving blue plane 1 while red plane 2 maneuvers behind blue plane 1 and successfully destroys it at t

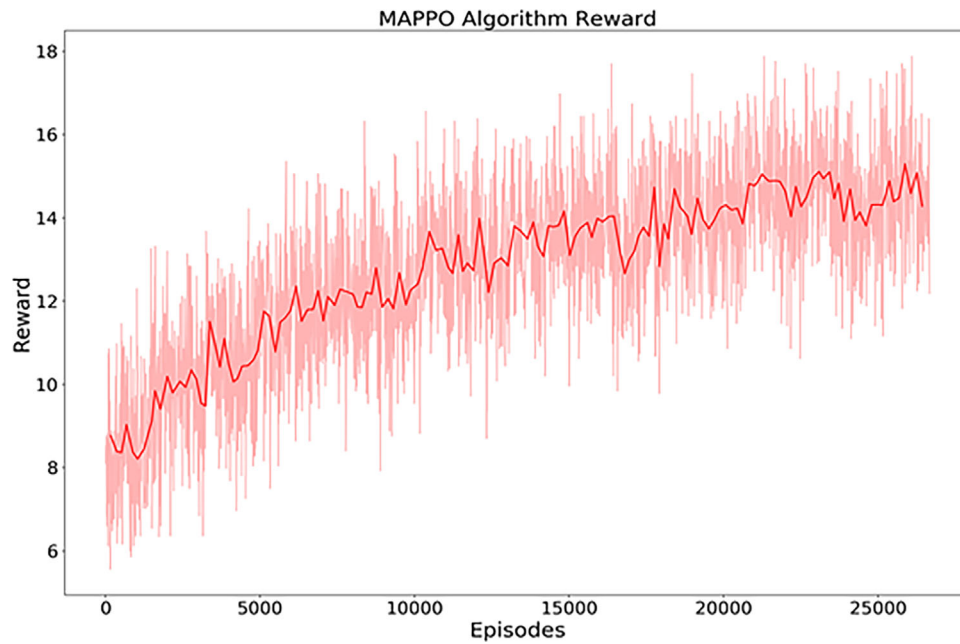


FIGURE 10 Average reward convergence figure in scenario 2.

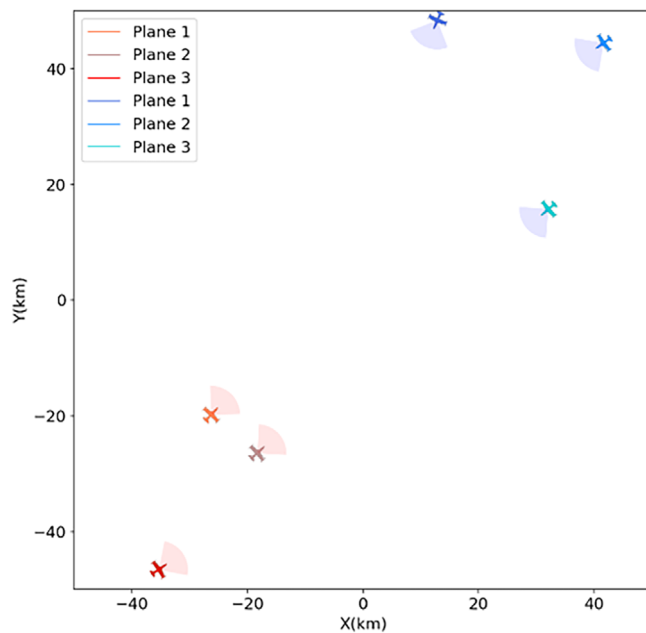


FIGURE 11 Situational of UAVs at the initial moment.

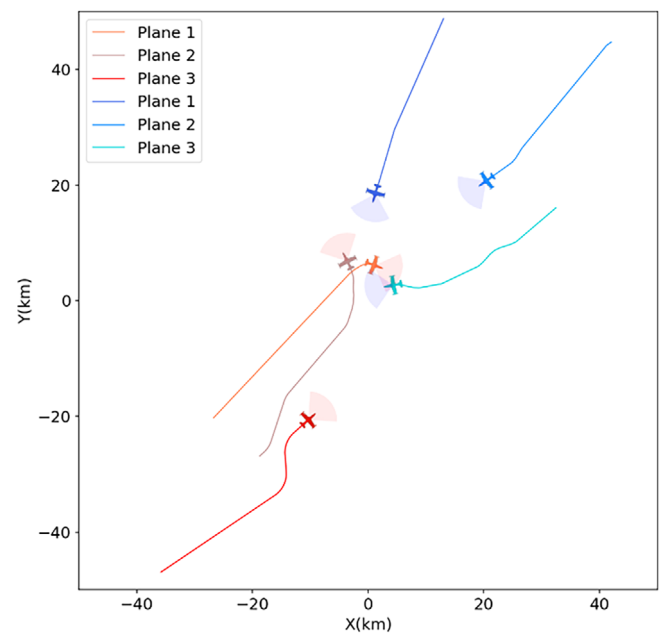


FIGURE 12 Maneuvering trajectories and situational at 65 s.

$= 82$ s. Subsequently, the red plane 3 enters the battlefield. The three red planes collaborate in their attack on the sole remaining blue plane, the blue plane 2. At $t = 118$ s, they effectively destroy the blue plane 2, securing victory for the red team's UAVs by eliminating all of the blue team's UAVs in the confrontation.

To further evaluate the performance of the proposed algorithm in Scenario 1 and Scenario 2, we calculated the average number of attacks, the average number of evasions, and the probability of eliminating all blue team's UAVs based on 50 sets of test data. The detailed results are shown in Table 6. From

the table, it can be observed that since the blue team's UAVs in Scenario 2 are more agile compared to those in Scenario 1, the algorithm attempts more attacks to destroy the blue team's UAVs and performs more evasions to avoid being destroyed. As a result, the probability of eliminating all blue team's UAVs in Scenario 2 still exceeds 50%.

To further demonstrate the effectiveness of the proposed algorithm, we extended the single-round game confrontation in Scenario 2 to a multi-round game confrontation. When all blue team's UAVs are eliminated, three new blue team's UAVs

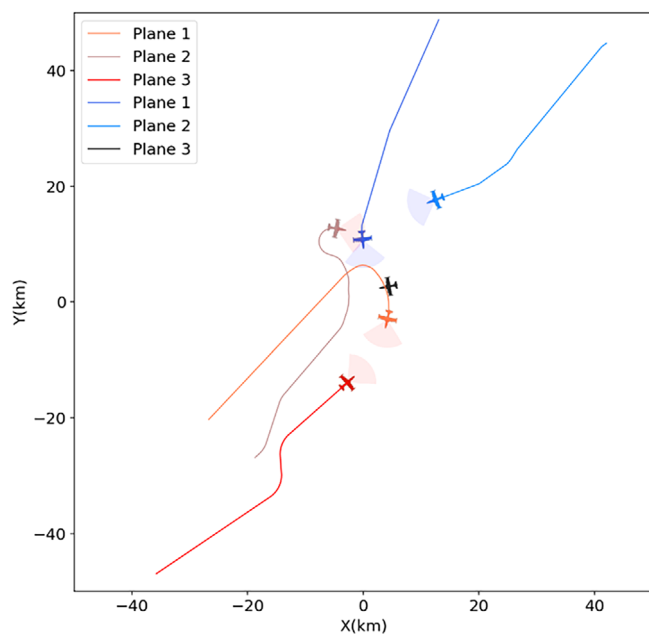


FIGURE 13 Maneuvering trajectories and situational at 82 s.

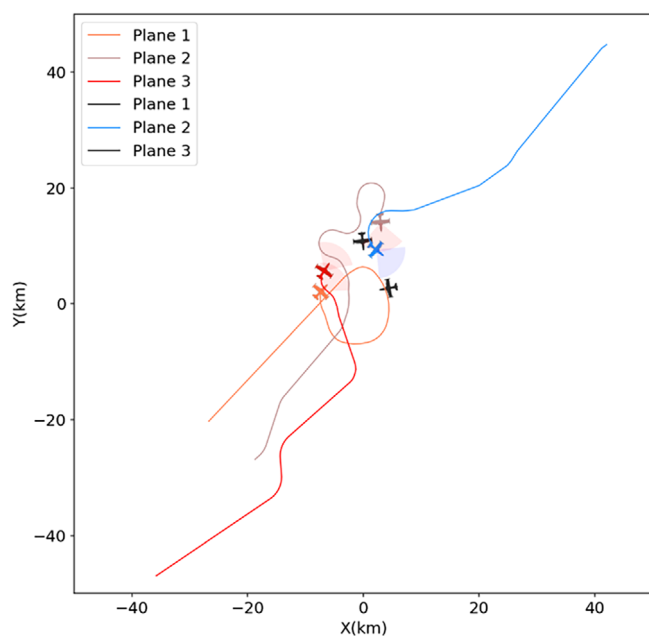


FIGURE 14 Maneuvering trajectories and situational at 118 s.

appear to continue the confrontation with the surviving red team's UAVs, until the episode length is reached.

Figures 16–20 show the maneuvering trajectories and real-time situational map of both the red and blue team's UAVs during the multi-round game confrontation.

In the depicted scenario (Figures 16–20), after all three blue team UAVs in the first round of confrontation were destroyed, the blue team deployed another three UAVs at $t = 111$ s to continue the game confrontation with the surviving red team UAVs. The surviving red team UAVs adjusted their direction

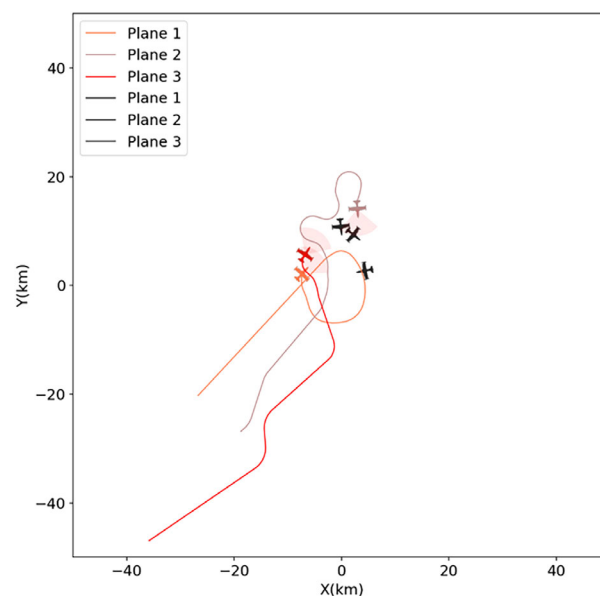


FIGURE 15 UAV adversarial outcome.

TABLE 6 Single-round game confrontation algorithm performance.

	Scenario 1	Scenario 2
Average attacks per round	3.8	4.4
Average evasions per round	2.7	3.2
Total elimination	70%	58%

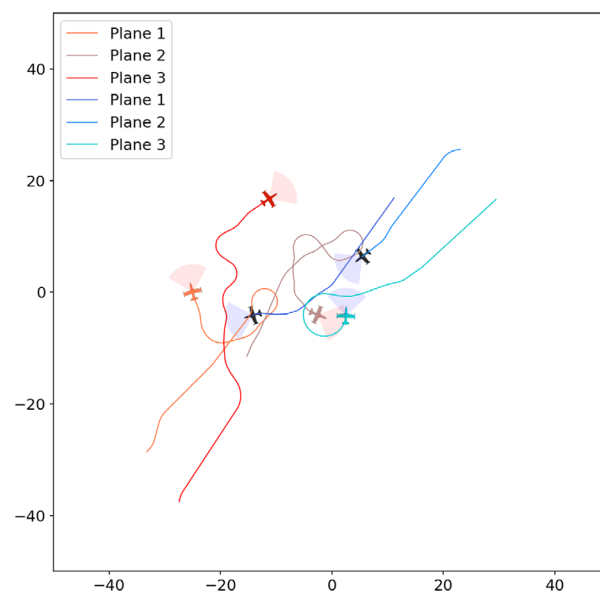


FIGURE 16 Defeat all blue team UAVs in the first round.

and flew towards the second round of blue team UAVs, successfully destroying two blue team UAVs at $t = 158$ s and $t = 178$ s. However, due to the proximity of the blue team's three UAVs, two red team UAVs were also destroyed by the third blue team UAV. The confrontation ended when the episode length

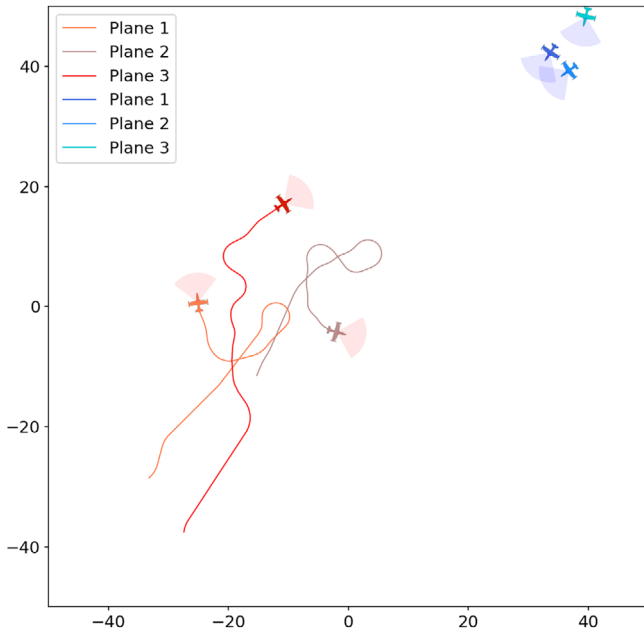


FIGURE 17 The second round of three blue team UAVs appears at the 111 s.

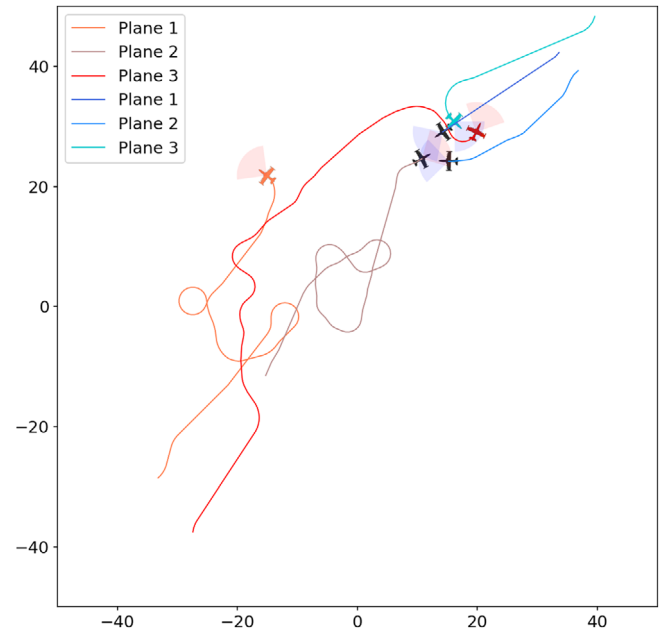


FIGURE 19 Maneuvering trajectories and situational at 178 s.

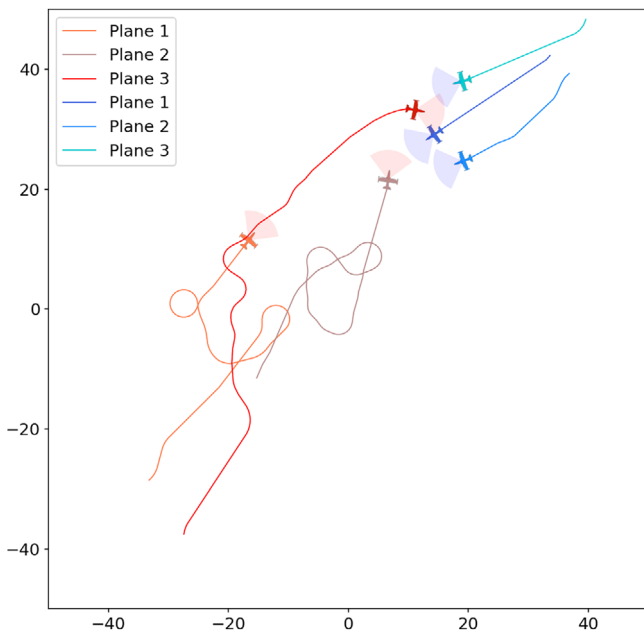


FIGURE 18 Maneuvering trajectories and situational at 158 s.

was reached, with the red team having destroyed five blue team UAVs at the cost of losing two of their own.

5 | CONCLUSION

This article tackles the challenges of autonomous collaborative real-time control decision-making in UAV swarm adversarial by creating a 3vs3 UAV swarm adversarial simulation environment. It introduces a UAV control decision method based

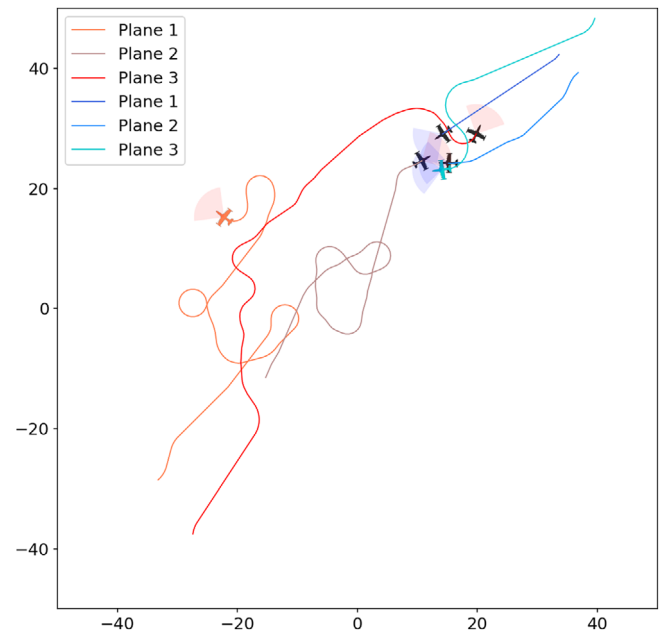


FIGURE 20 The episode length is reached.

on the MAPPO algorithm, along with corresponding environmental state representations and reward functions tailored for complex adversarial scenarios. The subsequent UAV swarm adversarial gaming simulations demonstrate the method's ability to facilitate the generation of effective collaborative effects and cooperative tactics, such as coordinated attacks and deceptive interferences, thereby enhancing decision-making performance. These findings lay the groundwork for autonomously generating maneuver decisions in more intricate and realistic adversarial scenarios in future research. The article's primary focus is to explore how effective swarm adversarial strategies,

closely aligned with practical application requirements, can be efficiently trained using intelligent state representations and straightforward reward function designs within complex UAV swarm adversarial scenarios. The upcoming phase of work will involve comprehensive research across three key areas: UAV modeling, adversarial scenario design, and advancements in multi-agent reinforcement learning methods.

AUTHOR CONTRIBUTIONS

Yongkang Jiao: Data curation; resources; software. **Wenxing Fu:** Formal analysis; writing—original draft; writing—review and editing. **Xinying Cao:** Writing—review and editing. **Qiangqing Su:** Software; validation; visualization. **Yusheng Wang:** Formal analysis; writing—original draft. **Zixiang Shen:** Software; validation; visualization; writing—review and editing. **Lanlin Yu:** Methodology; supervision; writing—review and editing.

ACKNOWLEDGEMENTS

The authors would sincerely thank the people who supported this work and the reviewing committee for their estimable feedback.

CONFLICT OF INTEREST STATEMENT

The authors declare no conflicts of interest.

DATA AVAILABILITY STATEMENT

The data supporting this study's results are available from the submitting author upon reasonable request.

ORCID

Lanlin Yu  <https://orcid.org/0000-0002-0664-2464>

REFERENCES

- Saska, M., Vonásek, V., Chudoba, J., Thomas, J., Loianno, G., Kumar, V.: Swarm distribution and deployment for cooperative surveillance by micro-aerial vehicles. *J. Intell. Rob. Syst.* 84, 469–492 (2016)
- Chen, X., Hu, N.: Effectiveness evaluation for UAV air-to-ground attack based on PSO-BP neural network. In: *IEEE Chinese Control Conference*, pp. 3864–3869. IEEE, Piscataway (2017)
- Ernest, N., Cohen, K., Kivelevitch, E., Schumacher, C., Casbeer, D.: Genetic fuzzy trees and their application towards autonomous training and control of a squadron of unmanned combat aerial vehicles. *Unmanned Syst.* 3(03), 185–204 (2015)
- Huang, H., Savkin, A.V.: An algorithm of reactive collision-free 3-D deployment of networked unmanned aerial vehicles for surveillance and monitoring. *IEEE Trans. Ind. Inf.* 16(1), 132–140 (2019)
- Jiang, X., Chen, X., Tang, J., et al.: Covert communication in UAV-assisted air-ground networks. *IEEE Wireless Commun.* 28(4), 190–197 (2021)
- Qirui, Z., Ruixuan, W.: Ground attack strategy of cooperative UAVs for multitargets. *Complexity* 2019, 1–13 (2019)
- Xu, G., Liu, Q., Zhang, H.: The application of situation function in differential game problem of the air combat. In: *IEEE Chinese Automation Congress*, pp. 1190–1195. IEEE, Piscataway (2018)
- Changqiang, H., Kangsheng, D., Hanqiao, H., Shangqin, T., Zhuoran, Z.: Autonomous air combat maneuver decision using Bayesian inference and moving horizon optimization. *J. Syst. Eng. Electron.* 29(1), 86–97 (2018)
- Hu, D., Yang, R., Zhang, Y., et al.: Aerial combat maneuvering policy learning based on confrontation demonstrations and dynamic quality replay. *Eng. Appl. Artif. Intell.* 111, 104767 (2022)
- Jiandong, Z., Qiming, Y., Guoqing, S., Yi, L., Yong, W.: UAV cooperative air combat maneuver decision based on multi-agent reinforcement learning. *J. Syst. Eng. Electron.* 32(6), 1421–1438 (2021)
- Teng, T.H., Tan, A.H., Tan, Y.S., Yeo, A.: Self-organizing neural networks for learning air combat maneuvers. In: *IEEE International Joint Conference on Neural Networks*, pp. 1–8. IEEE, Piscataway (2012)
- Fan, Z., Xu, Y., Kang, Y., Luo, D.: Air combat maneuver decision method based on A3C deep reinforcement learning. *Machines* 10(11), 1033 (2022)
- Li, S., Jia, Y., Yang, F., Qin, Q., Gao, H., Zhou, Y.: Collaborative decision-making method for multi-UAV based on multiagent reinforcement learning. *IEEE Access* 10, 91385–91396 (2022)
- Kong, W., Zhou, D., Du, Y., Zhou, Y., Zhao, Y.: Hierarchical multi-agent reinforcement learning for multi-aircraft close-range air combat. *IET Control Theory Appl.* 17(13), 1840–1862 (2023)
- Dey, S., Xu, H.: Hierarchical game theoretical distributed adaptive control for large scale multi-group multi-agent system. *IET Control Theory Appl.* 17(17), 2332–2352 (2023)
- Lin, Z., Ming'an, T., Wei, Z., Shengyun, Z.: Sequential maneuvering decisions based on multi-stage influence diagram in air combat. *J. Syst. Eng. Electron.* 18(3), 551–555 (2007)
- Li, S., Chen, M., Wang, Y., Wu, Q.: A fast algorithm to solve large-scale matrix games based on dimensionality reduction and its application in multiple unmanned combat air vehicles attack-defense decision-making. *Inf. Sci.* 594, 305–321 (2022)
- Hu, S., Ru, L., Lv, M., Wang, Z., Lu, B., Wang, W.: Evolutionary game analysis of behavior strategy for UAV swarm in communication-constrained environments. *IET Control Theory Appl.* 18(3), 350–363 (2024)
- Sun, W., Hao, M.: UAV Decision-making system based on the rough set theory and the optimal genetic algorithm. In: *IEEE*, pp. 209–213 (2020)
- Yu, Z., Zhang, Y., Liu, Z., Qu, Y., Su, C.Y.: Distributed adaptive fractional-order fault-tolerant cooperative control of networked unmanned aerial vehicles via fuzzy neural networks. *IET Control Theory Appl.* 13(17), 2917–2929 (2019)
- Lu, B., Ru, L., Lv, M., Hu, S., Zhang, H., Zhao, Z.: Enhanced LSTM-DQN algorithm for a two-player zero-sum game in three-dimensional space. *IET Control Theory Appl.* 18(18), 2798–2812 (2024)
- Venturini, F., Mason, F., Pese, F., et al.: Distributed reinforcement learning for flexible and efficient UAV swarm control. *IEEE Trans. Cognit. Commun. Network.* 7(3), 955–969 (2021)
- Ma, B., Liu, Z., Zhao, W., et al.: Target tracking control of UAV through deep reinforcement learning. *IEEE Trans. Intell. Transp. Syst.* 24(6), 5983–6000 (2023)
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms. *arXiv preprint, arXiv:1707.06347* (2017)
- Yu, C., Velu, A., Vinitzky, E., et al.: The surprising effectiveness of ppo in cooperative multi-agent games. In: *Advances in Neural Information Processing Systems*, vol. 35, pp. 24611–24624. Curran Associates, Inc., Red Hook, NY (2022)
- Yan, R., Li, S., Wang, C., et al.: Cooperative decision-making for heterogeneous UAV swarm confrontation based on self-play reinforcement learning. *Sci. Sin. Inf.* 54(7), 1709 (2024)

How to cite this article: Jiao, Y., Fu, W., Cao, X., Su, Q., Wang, Y., Shen, Z., Yu, L.: Collaborative decision-making for UAV swarm confrontation based on reinforcement learning. *IET Control Theory Appl.* 19, e12781 (2025). <https://doi.org/10.1049/cth2.12781>